

POSTER: Prediction-Enhanced Expert Prefetching and Eviction for MoE Offloading via PRED-MoE

Wenchen Han

University College London

Michael Mitzenmacher

Harvard University

Shay Vargaftik

VMware Research by Broadcom

Ran Ben Basat

University College London

ABSTRACT

Mixture-of-Experts (MoE) models improve scaling by activating a small number of experts per token. However, the combined memory requirements of all experts may exceed the available GPU high bandwidth memory (HBM) during serving. Inference frameworks such as vLLM and HuggingFace address the problem by offloading inactive experts to CPU memory and moving them to the GPU’s HBM as needed. While enabling inference of large models on limited HBM setups, this CPU-GPU traffic overhead slows token generation speed substantially.

We present PRED-MoE, a correctness-preserving prefetching and eviction policy for MoE offloading. PRED-MoE leverages a lightweight predictor to assess which experts are most likely to be required for prefetching and ranks experts by their likelihood of near-future for eviction. Preliminary evaluation over the Qwen3-30B-A3B model and the MMLU-Pro dataset indicates that PRED-MoE reduces the TPOT of vLLM and HuggingFace by up to 3.02× and 28.62× respectively.

1 INTRODUCTION

MoE layers route each token to a small expert subset, enabling larger parameter counts for the same HBM size [3, 5, 8]. The serving challenge is that expert parameters dominate memory: computation uses only a few experts at once, yet all experts must reside somewhere. When active experts do not fit in GPU memory, *MoE offloading* keeps a subset on GPU and the rest in CPU memory, loading experts on demand. This CPU-GPU traffic often dominates token generation.

This is naturally a prefetching-caching problem: prefetch experts likely to be useful soon, and evict experts to make room. Misses are costly because an MoE layer stalls until weights arrive over the CPU-GPU interconnect.

Prior systems show the promise of expert offloading, prefetching, and predictive routing [1, 2, 4], but eviction often overlooks autoregressive structure or changes system behavior, e.g., by modifying expert selection.

We profile expert activation patterns and build a real-time predictor of each expert’s near-future demand. PRED-MoE uses these predictions for prefetching and eviction while preserving correctness by serving exactly the router-selected experts and reducing CPU-GPU traffic.

In preliminary evaluation with Qwen3-30B-A3B [7] on MMLU-Pro [11], varying GPU memory and batch size up to 64, PRED-MoE outperforms vLLM [6] and HuggingFace [12] by 2.26 ~ 3.02× and 18.23 ~ 28.62×, respectively.

2 PRED-MOE DESIGN

2.1 Problem formulation

We formulate MoE offloading as online prefetching and caching. The GPU stores a subset of expert weights, CPU memory stores every expert, and PRED-MoE tries to keep the original router-selected experts resident on GPU when needed without changing routing.

Let $\mathcal{L}$ be the MoE layers. Each layer $\ell \in \mathcal{L}$ contains a disjoint expert set $\mathcal{E}_\ell$, and $\mathcal{X} = \bigcup_\ell \mathcal{E}_\ell$ is the full expert set. During autoregressive decoding, let $\mathcal{T}$ be the generation steps, $t \in \mathcal{T}$ one step, and $\mathcal{B}_t$ the active requests at step t .

For each $b \in \mathcal{B}_t$, layer ℓ , and step t , the router selects $\mathcal{A}_{b,t,\ell} \subseteq \mathcal{E}_\ell$; the batch-level active set is $\mathcal{A}_{t,\ell} = \bigcup_{b \in \mathcal{B}_t} \mathcal{A}_{b,t,\ell}$.

Let $\mathcal{C}_{t,\ell}^- \subseteq \mathcal{X}$ be the experts resident on GPU immediately before layer ℓ computes at step t , after prefetches but before loading required missing experts. Thus $\mathcal{A}_{t,\ell} \setminus \mathcal{C}_{t,\ell}^-$ must be loaded from CPU memory; such misses preserve correctness but add latency.

After the current layer’s expert computation begins, the system may prefetch experts predicted useful for future layer-step pairs (t', ℓ') . If the cache exceeds capacity C , eviction removes inactive resident experts; active experts cannot be replaced. Our goal is to minimize miss-induced transfer time:

$$\min_{\pi} \sum_{t \in \mathcal{T}} \sum_{\ell \in \mathcal{L}} \sum_{x \in \mathcal{A}_{t,\ell}} w_x \cdot (x \notin \mathcal{C}_{t,\ell}^-), \quad \text{s.t.} \quad |\mathcal{C}_{t,\ell}| \leq C.$$

Here, w_x is the blocking CPU-to-GPU transfer cost for expert x . Assuming equal-size experts and a common transfer path makes w_x constant, reducing the objective to minimizing expert misses.

The objective abstracts two lower-level effects. First, load costs may vary with expert size, GPU placement, bandwidth, and locality; nonuniform w_x can model this. Second, prefetched-but-unused experts are not charged separately, which is reasonable when bounded prefetch traffic overlaps computation. False prefetches still consume capacity and can increase future misses; we leave richer models to future work.

2.2 Expert activation patterns

PRED-MoE uses three MoE-routing properties, consistent with prior observations [10, 13]. First, each request and layer activates only a small, usually fixed-size subset: $\mathcal{A}_{b,t,\ell} \subseteq \mathcal{E}_\ell$ and $|\mathcal{A}_{b,t,\ell}| \ll |\mathcal{E}_\ell|$. Routing is also skewed: many batch requests select overlapping experts, so $\mathcal{A}_{t,\ell}$ is often much smaller than $\mathcal{E}_\ell$.

Second, decode steps exhibit temporal locality. For fixed layer ℓ , the active set at one generation step often overlaps nearby steps, making $\mathcal{A}_{t,\ell}$ predictive of future active sets. Thus recent expert usage helps eviction: recently used experts are more likely to be reused.

Third, expert choices are token- and context-dependent. Router decisions correlate with token representations and hence with request and generation-state features. PRED-MoE profiles these correlations to predict which experts are likely to be needed in upcoming layers or decode steps.

Together, these properties suggest a prediction-enhanced policy: keep experts likely to be reused soon, prefetch experts predicted for future layer-step pairs, and evict inactive experts with low reuse probability.

2.3 Prefetching and eviction

At each step t and MoE layer ℓ , PRED-MoE performs five steps. (1) Run the router and compute $\mathcal{A}_{t,\ell}$. (2) If capacity is insufficient for $\mathcal{A}_{t,\ell} \setminus C_{t,\ell}^-$, protect the current active set and evict the lowest-score inactive experts. (3) Admit missing active experts by CPU-to-GPU copy, overlapping copies with non-expert work when possible. (4) Update the routing trace and predictor features, using online training to preserve accuracy as workloads change. (5) While this layer processes the batch, prefetch experts for future steps/layers, choose eviction victims, and replace them with CPU-to-GPU copies.

3 EVALUATION

Our preliminary PRED-MoE implementation has two components: an MoE offloading engine atop vLLM [6] that manages inference-time prefetching-caching, and an offline-trained lightweight predictor that ranks experts by near-future demand.

Evaluation setup. We use Qwen3-30B-A3B, whose layers each have 128 experts and activate $K_l = 8$ experts per token,

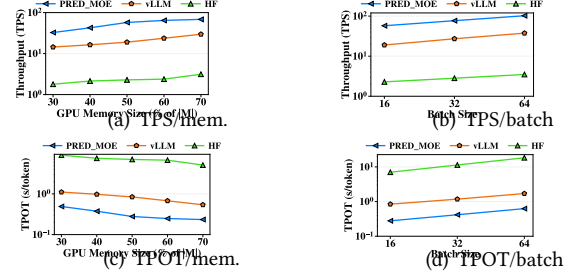


Figure 1: PRED-MoE’s throughput (TPS) and time per output token (TPOT) versus vLLM and HF. Panels (a,c) vary GPU memory at batch size 16; panels (b,d) vary batch size with 30GB GPU memory.

on MMLU-Pro [9, 11] with chain-of-thought prompting. Experiments run on a server with 8 NVIDIA RTX 6000 Ada GPUs (48 GB GDDR6 each), each connected to host CPU memory via PCIe 4.0 with 32 GB/s bidirectional bandwidth.

Training expert predictors. Predictors must be light enough for the critical path, so we restrict architecture and features. We train them offline from execution traces containing generated token embeddings and ground-truth routing logits, i.e., expert activations at every layer and decode step. Each training input uses token embeddings at t and $t-1$ plus routing outputs at $t-1$; the target is the experts selected at the prediction horizon. We train two predictors for $h \in \{0, 1\}$: $\mathcal{P}_{0+}$ for current-step activations and $\mathcal{P}_{1+}$ for next-step activations. Because each layer may activate multiple experts, we use an independent per-layer multi-label objective: the sum of binary cross-entropy terms between predicted expert probabilities and observed activation indicators. Each predictor is a small 171MB Transformer.

PRED-MoE policy at inference runtime. At the beginning of step t , for each request $b \in \mathcal{B}_t$, the predictors output per-layer scores $s_{b,\ell,e}^h$, the softmax-normalized likelihood that expert e activates at layer ℓ at step $t+h$.¹

Our initial prototype focuses on eviction and uses these scores in three phases. First, for each expert, it computes $p_{\ell,e,h=0}$ and $p_{\ell,e,h=1}$, the likelihoods that the expert is required at step $t+h$.² Second, it aggregates horizons as $\bar{P}_{\ell,e} = p_{\ell,e,0} + \lambda(1 - p_{\ell,e,0})p_{\ell,e,1}$, with $\lambda \geq 0$ tuned to reflect lower trust in $h = 1$ than $h = 0$. Third, at layer ℓ , when a cache miss occurs and the cache is full, PRED-MoE copies missing experts ($\mathcal{A}_{t,\ell} \setminus C_{t,\ell}^-$) from CPU to GPU and evicts non- $\mathcal{A}_{t,\ell}$ experts with the lowest $\bar{P}$ scores.

End to end inference performance. Figure 1 shows that PRED-MoE improves TPS over vLLM by 2.26 ~ 3.02 $\times$ and

¹Preliminary measurements show that running the predictors at inference adds less than 1% end-to-end time overhead.

²We max-pool across the batch to order experts, compute per-horizon priors P_h (expert is active | predicted rank r), and map ranks back to experts.

over HuggingFace by $18.23 \sim 25.18\times$ as GPU memory varies. As batch size varies, PRED-MoE improves TPOT by $2.74 \sim 3.02\times$ and $18.23 \sim 28.62\times$, respectively. The gains mainly come from fewer CPU-to-GPU copies: with batch size 16 and 50% GPU memory, PRED-MoE loads $\approx 2.2\%$ of experts on average, versus $\approx 10.63\%$ for vLLM and $\approx 50\%$ for HF.

4 FUTURE WORK

We plan to extend the prototype from prediction-guided eviction to full prefetching and eviction. This requires modeling cache misses, false prefetches, PCIe/NVLink and other interconnect bandwidth limits, and nonuniform expert-transfer costs.

REFERENCES

- [1] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. 2024. SiDA-MoE: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. *arXiv preprint arXiv:2310.18859* (2024). <https://arxiv.org/abs/2310.18859>
- [2] Artyom Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238* (2023). <https://arxiv.org/abs/2312.17238>
- [3] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. <https://jmlr.org/papers/v23/21-0998.html>
- [4] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. 2026. ExpertFlow: Efficient Mixture-of-Experts Inference via Predictive Expert Caching and Token Scheduling. *arXiv preprint arXiv:2410.17954* (2026). <https://arxiv.org/abs/2410.17954> Accepted at DAC 2026.
- [5] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bourgeois, Guillaume Lample, L  lio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Thibaut Gervet, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2024. Mixtral of Experts. *arXiv preprint arXiv:2401.04088* (2024). <https://arxiv.org/abs/2401.04088>
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*. <https://arxiv.org/abs/2309.06180>
- [7] Qwen Team. 2025. Qwen3-30B-A3B. <https://huggingface.co/Qwen/Qwen3-30B-A3B>. (2025). Hugging Face model repository.
- [8] Noam Shazeer, Azalia Mirhoseini, Krzysztof Mazi  r, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *arXiv preprint arXiv:1701.06538* (2017). <https://arxiv.org/abs/1701.06538>
- [9] TIGER-Lab. 2024. TIGER-Lab/MMLU-Pro. <https://huggingface.co/datasets/TIGER-Lab/MMLU-Pro>. (2024). Hugging Face dataset repository.
- [10] Hulin Wang, Yaqi Xia, Donglin Yang, Xiaobo Zhou, and Dazhao Cheng. 2025. Harnessing inter-gpu shared memory for seamless moe communication-computation fusion. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 170–182.
- [11] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *Advances in Neural Information Processing Systems* (2024). [arXiv:cs.CL/2406.01574](https://arxiv.org/abs/2406.01574) <https://arxiv.org/abs/2406.01574>
- [12] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, and Jamie Brew. 2020. HuggingFace’s Transformers: State-of-the-art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 38–45. <https://arxiv.org/abs/1910.03771>
- [13] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.